

Artificial Intelligence — Lecture 1

AI Fundamentals, Intelligent Agents, Environments, and Problem Formulation

Dr. Fayçal Hamdi

Faculty of Computer and Information Systems

Lecture 1 at a Glance

Dr. Fayçal Hamdi - For Course Use Only - 2026

AI Fundamentals, Intelligent Agents & Problem Formulation

Course Learning Outcome

CLO1: Describe fundamentals of AI including agents, problem state spaces, and the basic vocabulary used to formulate AI problems.

Lecture purpose

- Builds the conceptual base for search, CSPs, games, machine learning, NLP, and vision.
- Introduces AI through the rational-agent perspective from *Artificial Intelligence: A Modern Approach*.
- Connects intuitive examples with precise formal descriptions of agents, environments, and problems.

Conceptual structure

Stage	Core question
AI perspective	What does it mean for a system to behave intelligently?
Agent view	How do we describe perception, action, rationality, and environment?
Design view	How can an agent select actions and formulate problems?

💡 Main idea: AI systems can be studied as **agents** that perceive an **environment** and select **actions** to achieve goals.

Learning Objectives

Dr. Fayçal Hamdi - For Course Use Only - 2026

By the end of Lecture 1, you will be able to:

1. Explain the main views of Artificial Intelligence: human-centered and rationality-centered.
2. Describe an AI system using the **agent–environment** interaction model.
3. Analyze a task environment using **PEAS**: Performance, Environment, Actuators, Sensors.
4. Classify environments as observable, deterministic, episodic, static, discrete, and single-agent or multi-agent.
5. Compare common agent architectures: reflex, model-based, goal-based, utility-based, and learning agents.
6. Formulate a simple AI problem using states, actions, transition model, goal test, and path cost.

Lecture Roadmap

Dr. Fayçal Hamdi - For Course Use Only - 2026

From intuitive AI systems to formal problem descriptions

Stage	Main idea	Student output
1. AI perspectives	Different definitions of AI emphasize human behavior, human thought, rational action, or rational thought.	Explain why the rational-agent view is useful for engineering AI systems.
2. Agents and environments	An agent perceives its environment and acts according to a performance measure.	Draw and explain the agent–environment loop.
3. PEAS analysis	A task can be described by its Performance measure, Environment, Actuators, and Sensors.	Build a PEAS description for a realistic AI application.
4. Environment properties	The difficulty of AI design depends strongly on the environment type.	Classify an environment using standard properties.
5. Agent architectures	Different agent programs require different assumptions and internal representations.	Compare reflex, model-based, goal-based, utility-based, and learning agents.
6. Problem formulation	Many AI tasks can be represented as state-space problems.	Define states, actions, transition model, goal test, and path cost.

Part I

Dr. Fayçal Hamdi — Course Use Only - 2026

What is AI? Rational Agents & Task Environments

AI can be studied as the design of systems that **perceive, reason, learn, and act** in environments.

Dr. Fayçal Hamdi — For Course Use Only - 2026

What Does Artificial Intelligence Study?

AI is broader than a single tool or technology

AI studies computational systems that can perform tasks associated with intelligent behavior:

-  **Perceive** the world through data or sensors
-  **Represent** knowledge about objects, states, and relations
-  **Reason** about what is true or what should be done
-  **Learn** patterns from examples or experience
-  **Act** to achieve goals under constraints and uncertainty

Examples: route planning, medical decision support, chess programs, speech recognition, recommendation systems, autonomous robots, fraud detection, and language assistants.

Modern generative AI is only one part of AI. This course starts with the foundations: agents, search, constraints, games, and learning.

Four Perspectives on AI

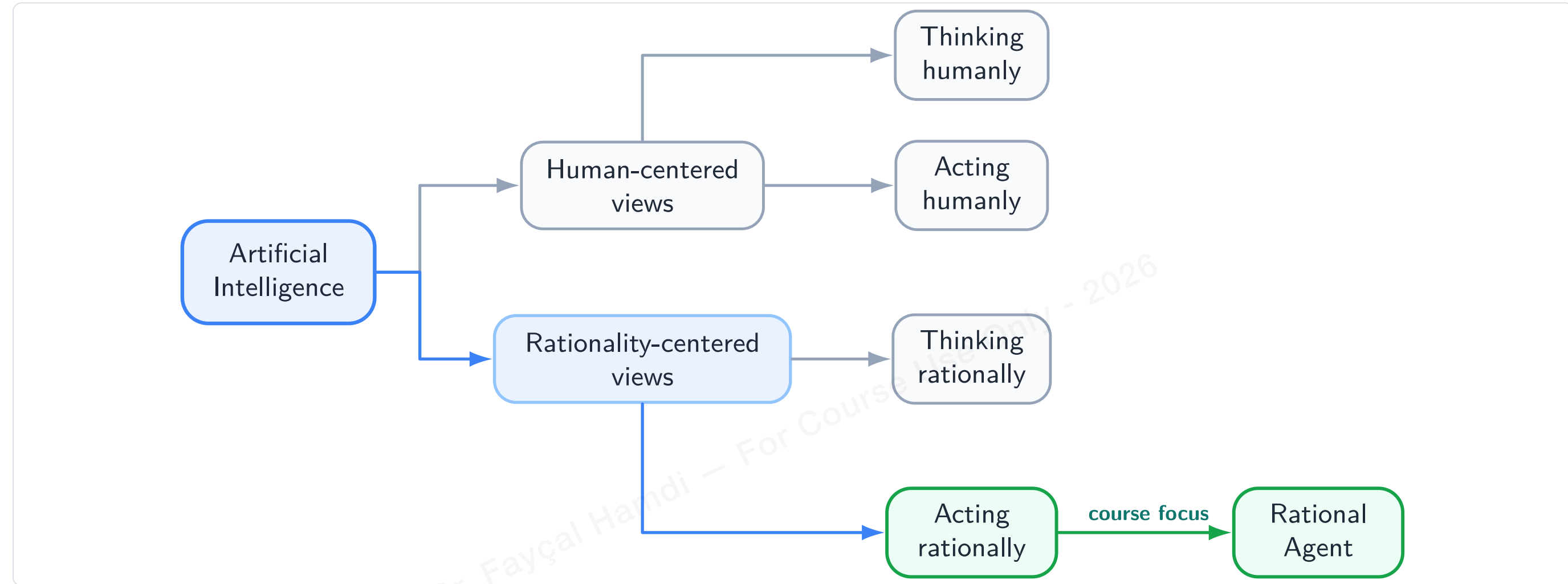
AIMA organizes AI definitions around two dimensions

Perspective	Main question	Typical method	Example concern
Thinking humanly	Does the system think like humans?	Cognitive modeling	Modeling human problem solving
Acting humanly	Does the system behave like humans?	Turing-test style evaluation	Human-like conversation
Thinking rationally	Does the system reason correctly?	Logic and inference	Valid conclusions from premises
Acting rationally	Does the system choose good actions?	Rational-agent design	Selecting actions that maximize performance

This course mainly uses the **acting rationally** view: we ask whether an agent chooses actions that are expected to achieve its goals, given what it has perceived and what it knows.

Perspective Map

From broad AI definitions to rational agents



The course adopts the **acting rationally** view: an AI system is studied as a **rational agent** that perceives, decides, and acts to achieve its objectives.

A Practical Definition for This Course

Dr. Fayçal Hamdi - Course Use Only - 2026

AI system

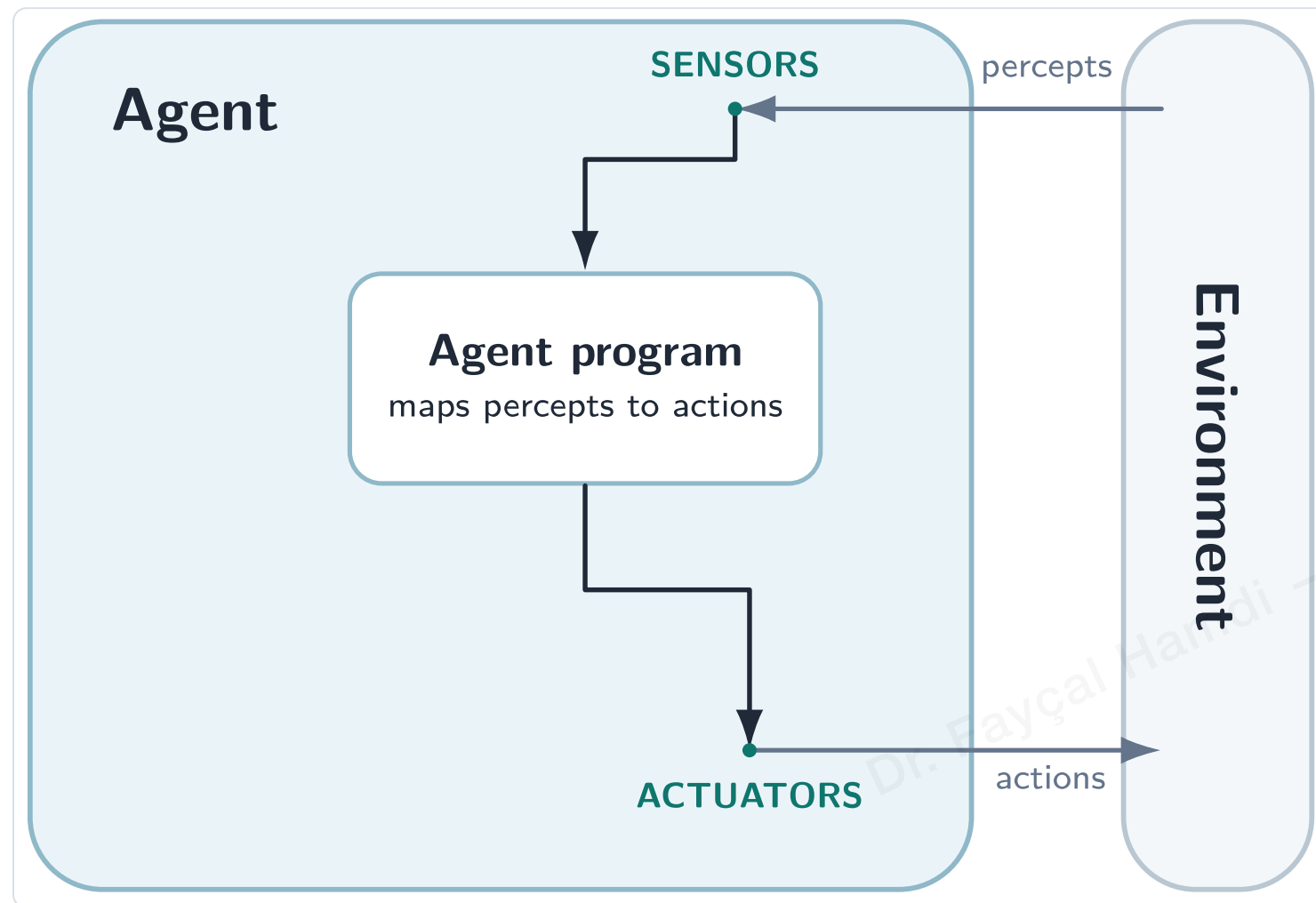
An **AI system** is a computational system that uses perception, representation, reasoning, learning, or action selection to perform tasks that require intelligent behavior.

This definition is intentionally broad. It includes symbolic AI systems, search algorithms, learning systems, natural language systems, computer vision systems, and embodied agents such as robots.

Course lens: Instead of asking whether a machine is truly “thinking,” we ask how its task can be formulated, what information it uses, how it evaluates success, and how it chooses actions.

The Agent–Environment Model

The central model of Lecture 1



How to read the figure

- **Environment** → world the agent interacts with
- **Sensors** → receive percepts
- **Agent program** → decides actions
- **Actuators** → execute actions

An **agent** perceives the environment through **sensors** and affects it through **actuators**.

The design question is concrete: what can the agent perceive, what can it do, and how will success be measured?

Agent Function and Agent Program

Two levels of description

Agent function

A mathematical mapping from percept history to action:

$$f : P^* \rightarrow A$$

- P^* : all possible percept sequences
- A : set of possible actions

This describes **what** the agent does.

Agent program

An implementation running on a physical or virtual architecture:

```
percept -> program -> action
```

This describes **how** the agent is implemented.

💡 Many different programs may implement the same agent function.

Rationality

Dr. Fayçal Hamdi - Course Use Only - 2026

Rational does not mean perfect

A **rational agent** selects the action expected to maximize its performance measure, given its percepts, knowledge, and available actions.

Rationality depends on

- performance measure;
- percept sequence;
- prior knowledge;
- available actions;
- computational limits.

Key point

Rationality is not omniscience. An agent can act rationally and still fail if the available information is incomplete or uncertain.

A delivery robot may choose a longer route because its map indicates a road closure.

Performance Measures

What counts as success?

A performance measure specifies how the agent's behavior is evaluated. It should focus on **outcomes in the environment**, not the internal appearance of intelligence.

Agent	Weak performance measure	Better performance measure
Vacuum agent	Moves frequently	Clean floor, low energy use, no damage
Route planner	Finds any route	Short, safe, low-cost route
Medical assistant	Gives many answers	Accurate, safe, explainable recommendations
Game-playing agent	Moves quickly	High probability of winning under time limits

Poor performance measures can produce undesirable behavior. AI design begins by asking what should be optimized.

PEAS: Describing a Task Environment

Performance, Environment, Actuators, Sensors

PEAS = $\langle$ Performance, Environment, Actuators, Sensors $\rangle$

PEAS is a compact way to describe the task the agent is designed to solve.

- **Performance:** how success is measured.
- **Environment:** where the agent operates.
- **Actuators:** how the agent acts.
- **Sensors:** how the agent receives information.

PEAS should be written before designing the algorithm. It clarifies the problem before solution details are discussed.

PEAS Example 1: Autonomous Taxi

Dr. Fayçal Hamdi - For Course Use Only - 2026

A complete task description

PEAS element	Example content
Performance	Safety, legality, passenger comfort, time, fuel efficiency, profit
Environment	Roads, traffic, pedestrians, signs, passengers, weather, maps
Actuators	Steering, accelerator, brakes, horn, turn signals, display/speech system
Sensors	Cameras, lidar/radar, GPS, speedometer, maps, microphones, engine sensors

A taxi agent must balance several objectives. A route that is fast but unsafe is not a good solution if safety is part of the performance measure.

PEAS Example 2: University Advising Chatbot

Not all agents are robots

PEAS element	Example content
Performance	Correctness, helpfulness, policy compliance, response time, student satisfaction
Environment	Students, course catalog, academic regulations, registration system
Actuators	Text responses, links, forms, escalation to human advisor
Sensors	Student questions, profile data, transcript, catalog updates, conversation history

Software agents also have environments. Their sensors and actuators are often data interfaces rather than physical devices.

Classroom Activity: Build a PEAS Description

Work in pairs — 7 minutes

Choose one of the following agents and complete a PEAS table:

- Spam-filtering email agent
- Chess-playing agent
- Smart irrigation system
- Robot vacuum cleaner
- Medical diagnosis assistant
- Exam proctoring system
- Recommendation engine
- Warehouse robot

Deliverable: one PEAS table with at least two items in each column. Be ready to explain one design trade-off in the performance measure.

Environment Properties

Why environment classification matters

The same agent design **does not work equally well** in every environment. Classification helps us predict how difficult the task is and what kind of algorithm may be needed

Property	Easier case	Harder case	Question to ask
Observability	Fully observable	Partially observable	Can the agent see the complete state?
Determinism	Deterministic	Stochastic	Is the next state predictable?
Episodicity	Episodic	Sequential	Do current actions affect future decisions?
Dynamics	Static	Dynamic	Can the world change while the agent deliberates?
Discreteness	Discrete	Continuous	Are states/actions/time countable or continuous?
Agents	Single-agent	Multi-agent	Are other decision makers present?

Environment Properties — Explanations

Each property affects design choices

Observability

If the agent cannot observe the complete state, it may need memory or probability.

Determinism

If actions have uncertain effects, the agent may need contingency plans.

Episodicity

If one action affects later options, planning becomes important.

Dynamics

If the environment changes during thinking time, the agent must respond quickly.

Discreteness

Continuous spaces often require approximation or numerical methods.

Multi-agent interaction

Other agents may cooperate, compete, or behave unpredictably.

💡 Environment classification is not just vocabulary. It tells us why some AI problems are much harder than others.

Worked Classification

Dr. Fayçal Hamdi - Course Use Only - 2026

Compare three task environments

Task	Observable?	Deterministic?	Sequential?	Dynamic?	Multi-agent?
Crossword puzzle	Mostly full	Deterministic	Sequential	Static	Single-agent
Autonomous taxi	Partial	Stochastic	Sequential	Dynamic	Multi-agent
Chess	Full	Deterministic	Sequential	Semi-static	Multi-agent

These labels are sometimes approximate. For example, chess is static while waiting for your move, but time pressure makes it semi-static because the clock changes.

Mini-Quiz

Dr. Fayçal Hamdi - For Course Use Only - 2026

Classify the environment

For a **medical diagnosis assistant**, decide whether the environment is:

1. fully observable or partially observable;
2. deterministic or stochastic;
3. episodic or sequential;
4. static or dynamic;
5. single-agent or multi-agent.

Likely answer: partially observable, stochastic, sequential, dynamic in many contexts, and multi-agent because patients, doctors, hospitals, and policies influence outcomes.

Part II

Dr. Fayçal Hamdi - Course Use Only - 2026

Agent Architectures & Problem Formulation

After describing an agent and its environment, we ask **how the agent can choose actions** and **how the task can be formalized** as a problem.

Dr. Fayçal Hamdi — For Course Use Only - 2026

From Agent Description to Agent Design

Two guiding questions

Description question

How do we describe the agent and its task environment?

- agent–environment loop
- rationality
- performance measure
- PEAS
- environment properties

Design question

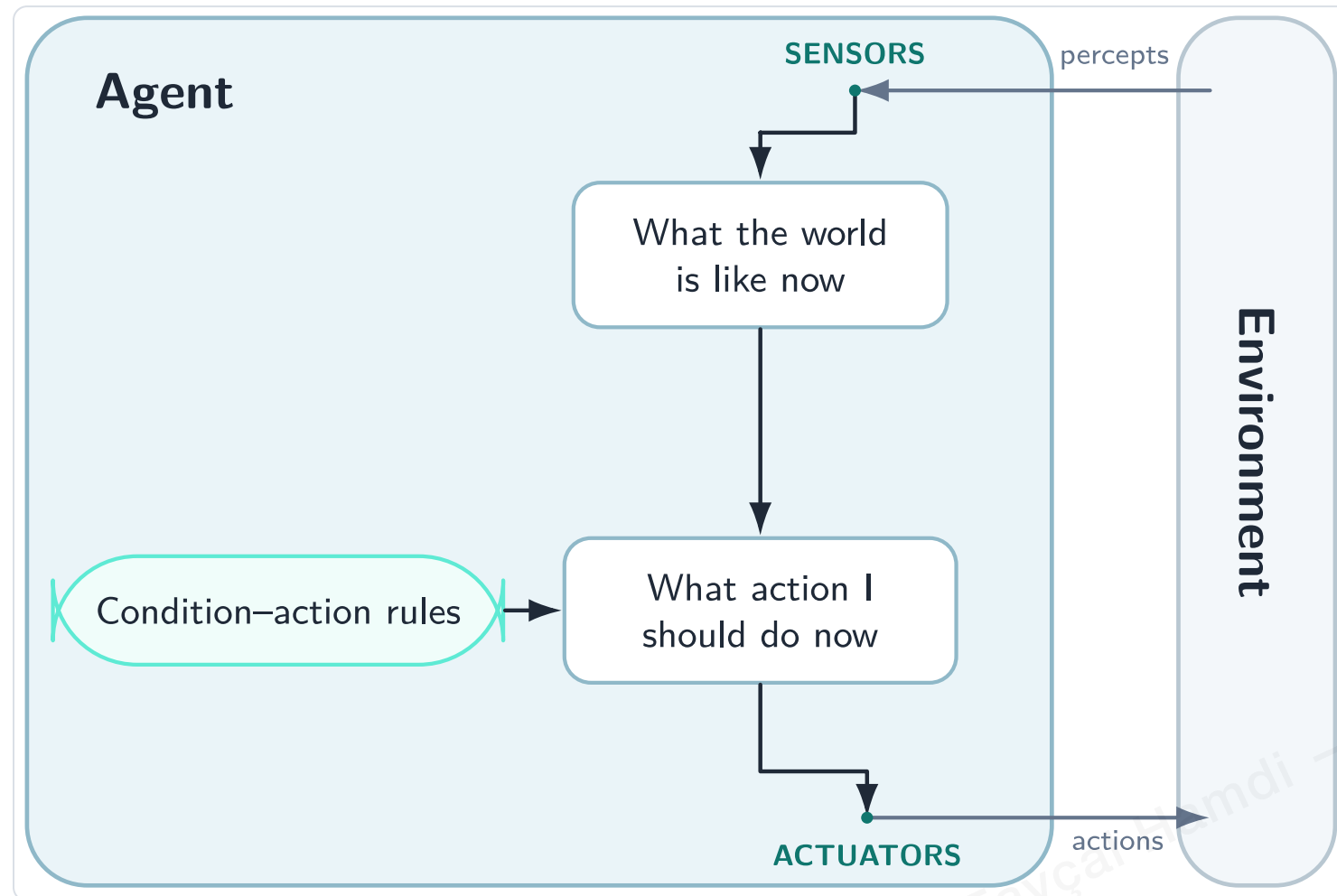
How can the agent select actions?

- agent architecture
- internal state or model
- goals and utilities
- problem formulation
- search space representation

💡 The move from description to design is the move from “What is the task?” to “What information and algorithm does the agent need?”

Simple Reflex Agents

Action depends only on the current percept



How to read the figure

- **Sensors** provide the current percept.
- **Condition-action rules** choose the response.
- **Actuators** execute the selected action.

A **simple reflex agent** selects an action from the current percept alone.

It works best when the environment is **fully observable**.

A simple reflex agent uses rules of the form: **if** condition **then** action

Example: If the vacuum sensor says the current square is dirty, then **Suck**; otherwise choose a movement action.

Simple Reflex Agent — Pseudocode

Rule-based action selection

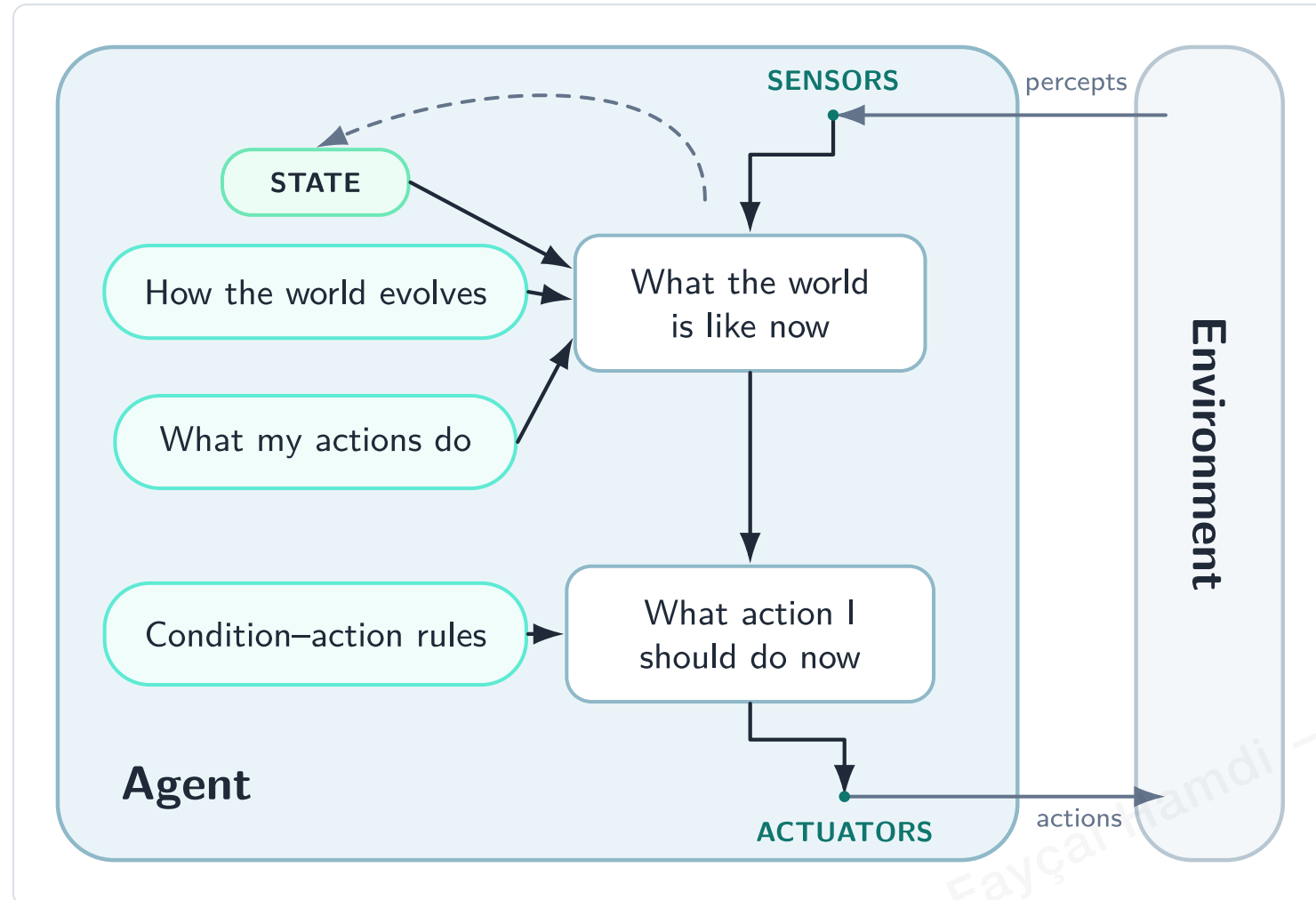
```
def simple_reflex_agent(percept):  
    state = interpret_input(percept)  
    rule = match_rule(state, rules)  
    return rule.action
```

The agent is fast and easy to implement because it does not maintain a detailed internal model. However, it assumes the current percept contains enough information to choose a good action.

💡 Simple reflex agents are appropriate mainly for fully observable tasks with reliable rules.

Model-Based Reflex Agents

Use internal state to handle partial observability



What is added here?

- The agent maintains an internal **state**.
- It uses a model of **how the world evolves**.
- It models **what its actions do**.

A **model-based reflex agent** acts using both the current percept and an internal model of the world.

Example: A robot vacuum can remember rooms already cleaned even when they are no longer visible.

Model-Based Reflex Agent — Pseudocode

Updating an internal representation of the world

```
def model_based_reflex_agent(percept):
    # Update the current state
    state = update_state(
        state,
        previous_action,
        percept,
        transition_model,
        sensor_model
    )
    # Match the state against condition-action rules
    rule = rule_match(state, rules)
    # Select and execute the action
    action = rule.action
    # Remember what we did for the next cycle
    previous_action = action
    return action
```

Persistent information

- `state` summarizes what the agent believes about the world.
- `previous_action` records what the agent did previously.

Knowledge used

- `transition_model`: how the world changes;
- `sensor_model`: how world states generate percepts.

💡 The crucial difference is **memory**:

Simple reflex: current percept → action

Model-based reflex: previous state + action + new percept → updated state → action

Goal-Based Agents



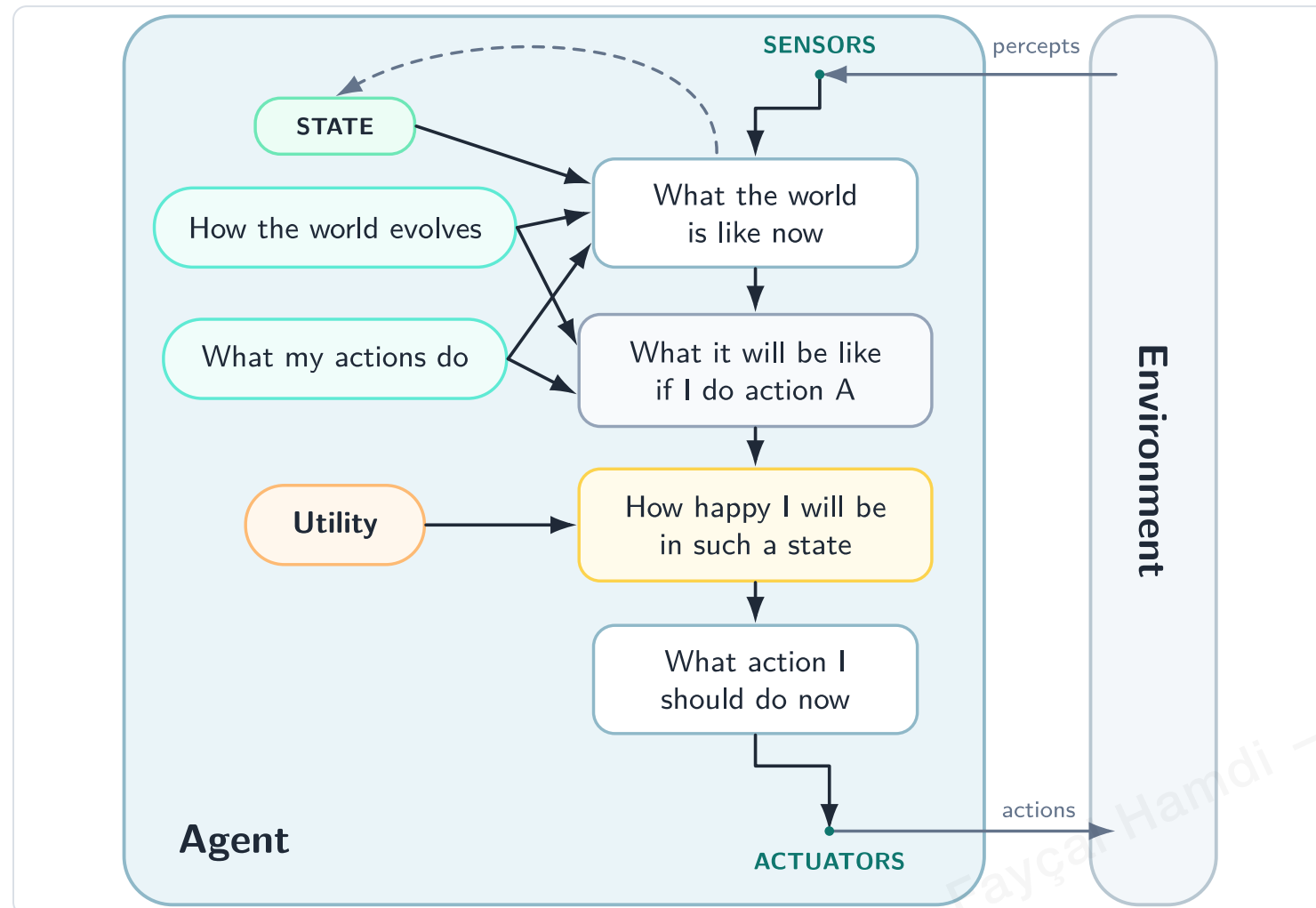
- The agent keeps a state and a model.
- It predicts the result of actions.
- It selects actions that lead toward goals.

A **goal** specifies a set of desirable states. Search and planning can then identify an action sequence that reaches them.

Example: A route-planning agent chooses actions that lead from its current location to the destination.

Utility-Based Agents

When there are many acceptable outcomes



Main idea

- Predict the result of each possible action.
- Evaluate the resulting state using utility.
- Prefer the action with the highest expected utility.

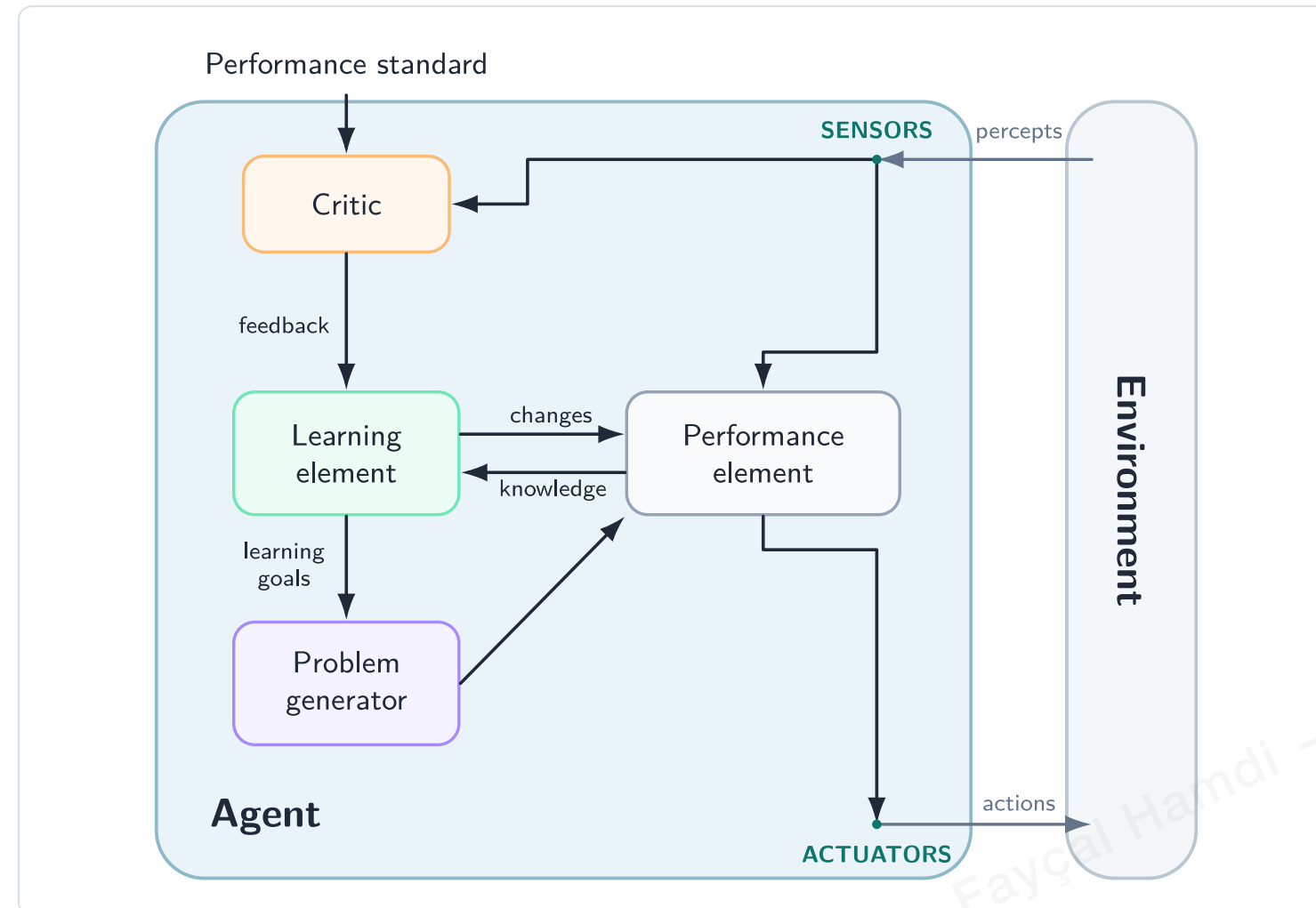
A **utility function** assigns a numerical value to states or outcomes:

$$U(s) \in \mathbb{R}$$

Example: A route planner may prefer a 20-minute safe route over an 18-minute risky route.

Learning Agents

Improve behavior from experience



Components in the diagram

- The **performance element** chooses actions.
- The **critic** evaluates behavior.
- The **learning element** improves the agent.
- The **problem generator** encourages exploration.

A **learning agent** improves its behavior through feedback from experience.

Example: A recommendation agent improves its suggestions by learning from user feedback.

Comparing Agent Architectures

Dr. Fayçal Hamdi - Course Use Only - 2026

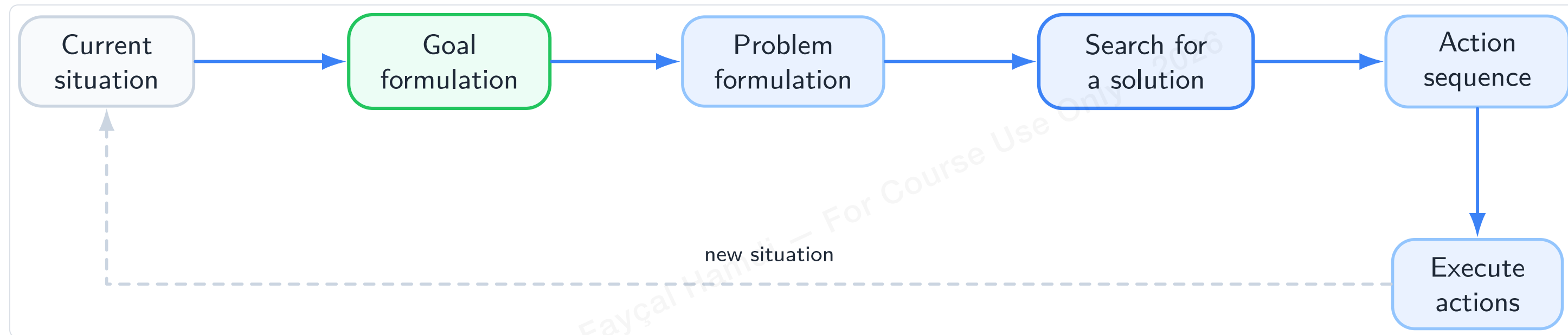
What information does each architecture use?

Architecture	Main idea	Strength	Limitation
Simple reflex	Current percept → rule → action	Fast and simple	Poor under partial observability
Model-based reflex	Maintain internal state	Handles hidden information	Requires a model of the world
Goal-based	Search for goal-achieving actions	Flexible and purposeful	Can be computationally expensive
Utility-based	Maximize preference/utility	Handles trade-offs	Utility may be hard to define
Learning agent	Improve from feedback	Adapts over time	Needs data, evaluation, and safeguards

From Agents to Problem-Solving

A problem-solving agent uses goals to choose action sequences

The agent first formulates a goal, then formulates a problem, then searches for an action sequence that solves the problem.



A **solution** is usually a sequence of actions that transforms the initial state into a goal state with acceptable cost.

Formal Problem Formulation

Dr. Fayçal Hamdi - Course Use Only - 2026

The basic state-space model

$$P = (S, A, s_0, T, G, c)$$

Symbol	Meaning	Question
S	State space	What situations can the agent be in?
A	Actions	What can the agent do?
s_0	Initial state	Where does the problem start?
T	Transition model	What state results from each action?
G	Goal test / goal states	How do we know the problem is solved?
c	Path or step cost	How expensive is a solution?

States vs. Real World

A state is an abstraction

A state does not include every physical detail of the world. It includes the information needed for decision-making.

Real world

A campus navigation problem includes weather, buildings, people, traffic, lighting, signs, construction, and many small details.

State-space abstraction

A useful state might only include:

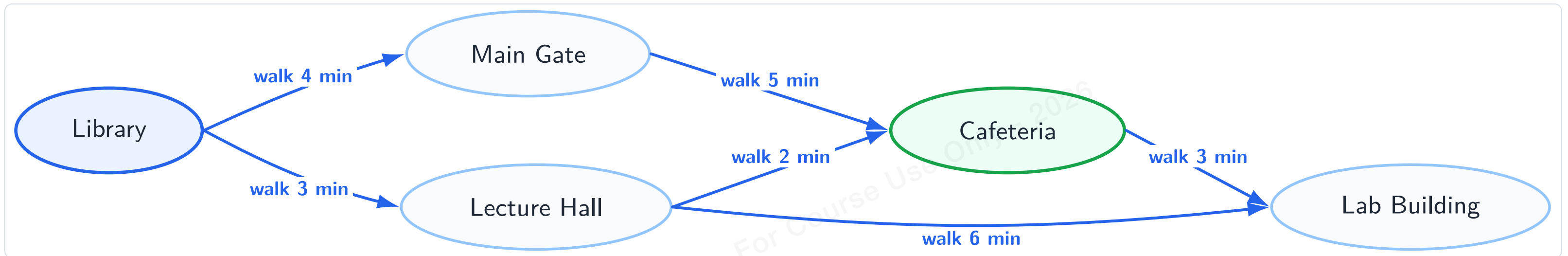
- current location;
- destination;
- available paths;
- estimated travel cost.

Good abstraction is essential. Too much detail makes the problem difficult; too little detail can remove information needed for correct decisions.

State-Space Graph

Nodes are states; edges are actions

A **state-space graph** represents the possible situations of a problem and the actions that connect them.



💡 A search algorithm explores this state space to find a **solution path**. If actions have costs, it may also seek the **lowest-cost path**.

Search Tree vs. State-Space Graph

Why repeated states matter

State-space graph

- Represents each distinct state once.
- Shows the actual structure of the problem.
- Can contain cycles.

Search tree

- Represents paths explored by the algorithm.
- The same state can appear multiple times through different paths.
- Useful for analyzing algorithm behavior.

If a robot can move from room A to room B and back to A, a naïve search tree can expand repeated states forever unless the algorithm detects already explored states.

Worked Example: Vacuum World

Formulating a simple AI problem

Consider a two-location vacuum world with locations **A** and **B**. Each location may be clean or dirty, and the agent is in one of the two locations.

Component	Formulation
States	Agent location plus cleanliness of A and B
Initial state	Example: agent at A, A dirty, B dirty
Actions	Left, Right, Suck, NoOp
Transition model	Suck cleans current square; Left/Right changes location if possible
Goal test	Both locations are clean
Cost	Often one unit per action

Vacuum World State Count

Counting helps reveal the size of a state space

The vacuum world has:

- 2 possible agent locations: A or B;
- 2 possible cleanliness values for A: clean or dirty;
- 2 possible cleanliness values for B: clean or dirty.

$$|S| = 2 \times 2 \times 2 = 8$$

💡 Even tiny problems can be represented formally. Larger AI problems quickly produce very large state spaces, which is why search strategy matters.

Worked Example: 8-Puzzle

Dr. Fayçal Hamdi - Course Use Only - 2026

A classical state-space problem

The 8-puzzle consists of numbered tiles and one blank space. The objective is to reach a goal arrangement by sliding tiles into the blank position.

Component	Formulation
States	All possible tile configurations
Initial state	Given starting arrangement
Actions	Move blank Up, Down, Left, Right when legal
Transition model	Swap blank with adjacent tile
Goal test	Configuration equals target arrangement
Cost	Usually one per move

This problem is ideal for demonstrating search because states, actions, paths, and costs are easy to visualize.

Problem Formulation Checklist

Dr. Fayçal Hamdi - Course Use Only - 2026

Use this template for any search problem

1. **Define states:** What information describes a situation sufficiently?
2. **Define the initial state:** Where does the agent start?
3. **Define actions:** What actions are legal in each state?
4. **Define the transition model:** What happens after each action?
5. **Define the goal test:** How do we recognize success?
6. **Define the cost function:** What makes one solution better than another?

Do not start by choosing an algorithm. First define the problem. An unclear formulation leads to unclear solutions.

Short Activity: Formulate a Problem

Work in groups — 10 minutes

Formulate one of the following as a state-space problem:

- A robot moving between rooms
- A route planner between campus buildings
- A student scheduling courses
- A puzzle game
- A delivery drone task
- A warehouse picking robot

Deliverable: Write S, A, s_0, T, G, c . Then explain one abstraction choice: what information did you include, and what did you ignore?

Common Formulation Mistakes

What to avoid

Mistake	Why it is a problem	Better practice
States are too vague	Algorithm cannot operate on them	Define precise variables
Actions are not legal-state dependent	Some moves may be impossible	Specify applicable actions
Goal test is ambiguous	The solution cannot be verified	Use a clear Boolean condition
Cost is ignored	Cannot compare solutions	Define unit cost or meaningful cost
Real-world details are excessive	State space becomes too large	Abstract only relevant information

Connecting Lecture 1 to Lecture 2

Why problem formulation comes before search

Once a problem is formulated, the state space can be explored using **different families of search methods**.

Problem formulation

$$P = (S, A, s_0, T, G, c)$$

Defines the **states, actions, transition model, goal test, and cost**.

Lecture 2: two main search families

Uninformed search

Explores the state space **without heuristic guidance**.

- Breadth-First Search
- Depth-First Search
- Uniform-Cost Search
- Iterative Deepening Search

Informed search

Uses **heuristic information** to guide exploration.

- Greedy Best-First Search
- A* Search
- Heuristic quality

💡 Lecture 2 will compare both families using **completeness, optimality, time complexity, and space complexity**.

Lecture 1 Summary

Dr. Fayçal Hamdi — For Course Use Only - 2026

Key ideas to retain

Conceptual foundations

- AI studies intelligent behavior in computational systems.
- A rational agent selects actions expected to maximize performance.
- PEAS clarifies the task before the algorithm is chosen.
- Environment properties influence the difficulty of agent design.

Formal foundations

- Agent function: $f : P^* \rightarrow A$
- Problem model: $P = (S, A, s_0, T, G, c)$
- A state is an abstraction of the real situation.
- A solution is a path from the initial state to a goal state.

End-of-Lecture Check

Five questions to check your understanding

1. What is the difference between an agent function and an agent program?
2. Why is rationality evaluated relative to percepts, knowledge, and available actions?
3. How does PEAS help before algorithm design begins?
4. Why can two agents in the same environment require different performance measures?
5. How do we formulate a task as a state-space problem?

Preparation for next lecture: Review how a state-space graph represents possible actions. Lecture 2 begins with uninformed search and the concept of the frontier.

Suggested Reading

Dr. Fayçal Hamdi - Course Use Only - 2026

Primary textbook

S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, Prentice Hall, 2020.

- Chapter 1: Introduction
- Chapter 2: Intelligent Agents
- Chapter 3: Solving Problems by Searching — introductory problem formulation sections

💡 Focus on the agent view, PEAS task environments, rationality, and the formal elements of problem formulation.